

Práctica 1: Repaso de Java e Introducción a la programación Multihilos

Alumna: Montes de Oca Aguilar Yessica Vianney

1. Linus no considera que “hilo” y “proceso” sean dos cosas separadas diferentes y propone pensar todo en un solo contexto de ejecución: conjunto total del estado de una tarea (registros de CPU, mapeo de memoria, archivos abiertos, etc.). Se relaciona porque la definición de la práctica sí distingue explícitamente hilo y proceso.

2. 12

```
jshell> Runtime.getRuntime().availableProcessors()
$1 ==> 12
```

3. Tardó 875,300 ns (≈ 0.875 ms)

```
C:\Users\Vianney\Downloads>java unam/fc/concurrent/practical/DeterminanteConcurrente
Program took 875300ns, result: -18
```

4. Tardó 2,109,400 ns (≈ 2.109 ms)

```
C:\Users\Vianney\Downloads>java unam/fc/concurrent/practical/DeterminanteConcurrenteRunnable
Program took 2109400ns, result: -18
```

DeterminanteConcurrenteRunnable.java:

Programa que calcula el determinante de una matriz 3x3 implementando la interfaz Runnable (en vez de extender Thread). Crea 6 objetos Runnable, cada uno envuelto en un objeto Thread distinto, donde cada hilo calcula una de las 6 multiplicaciones de la regla de Sarrus; al final se suman/restan los resultados parciales para obtener el determinante.

5. Tardó 5,000 ns (≈ 0.005 ms)

```
C:\Users\Vianney\Downloads>java unam/fc/concurrent/practical/DeterminanteSecuencial
Program took 5000ns, result: -18
```

DeterminanteSecuencial.java:

Programa que calcula el determinante de una matriz 3x3 de forma secuencial, sin utilizar hilos. Realiza las 6 multiplicaciones y la suma/resta final directamente en el método main().

6. Tardó 651,200 ns (≈ 0.651 ms)

```
C:\Users\Vianney\Downloads>java unam/fc/concurrent/practical/DeterminanteDosHilos
Program took 651200ns, result: -18
```

DeterminanteDosHilos.java:

Programa que calcula el determinante de una matriz 3x3 utilizando solo 2 hilos (en vez de 6). Cada hilo calcula 3 multiplicaciones (un hilo las diagonales que se suman, el otro las que se restan), y al final se combinan los dos resultados parciales para obtener el determinante.

7.

Versión	Tiempo (ns)	Hilos
Secuencial	5,000	0
2 hilos	651,200	2
6 hilos (Thread)	875,300	6
6 hilos (Runnable)	2,109,400	6

El orden va de menor a mayor conforme aumenta el número de hilos creados porque crear, arrancar y esperar con `join()` un hilo tiene un **costo de overhead** (asignar stack, registrar el hilo en el scheduler, cambios de contexto). Como el trabajo por hilo es pequeño; entre más hilos se crean más overhead se acumula sin ganar nada en paralelismo, por eso la secuencial (0 overhead) es la más rápida. Entre las dos versiones con el mismo número de 6 hilos, Runnable es la más lenta porque además de la creación de los 6 Runnable se tiene que envolver cada uno en un objeto Thread aparte.

8. Como el secuencial (5,000 ns) es mucho más rápido que el de 2 hilos (651,200 ns), el speedup real es **menor a 1**. De hecho el programa con hilos es ~130 veces más *lento*, no más rápido. Esto pasa porque la tarea es demasiado pequeña para justificar el overhead de crear hilos; Amdahl asume que el paralelismo *ayuda* en la parte paralelizable pero aquí el costo de paralelizar supera cualquier beneficio.

9. **qué hace `join()`:**

`join()` hace que el hilo que lo invoca (normalmente el hilo principal) se detenga y espere a que el hilo sobre el cual se llama termine su ejecución antes de continuar.

¿sigue funcionando sin `join()` en Determinante Concurrente?

El programa podría seguir dando el resultado correcto la mayoría de las veces (porque las multiplicaciones son tan rápidas que probablemente ya terminaron antes de que el hilo principal llegue a sumarlas) pero es una condición de carrera: sin `join()` el hilo principal podría intentar leer `thr1.partial`, `thr2.partial`, etc. antes de que esos hilos hayan terminado de calcularlos dando resultados incorrectos (probablemente 0 que es su valor inicial) de forma intermitente e impredecible.